Virtual Threads

Virtual Threads · Fan-out 10k blocking HTTP checks · Build with Waqar

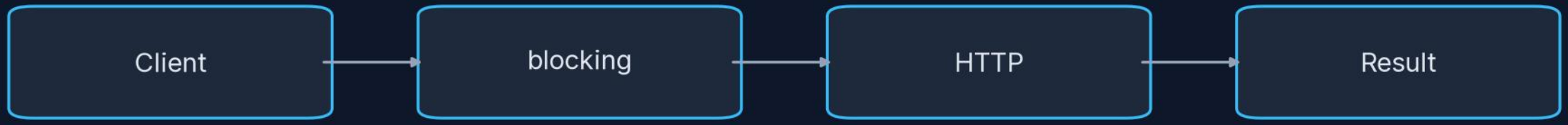
Today's Hook

- Problem: Fan-out 10k blocking HTTP checks
- Pattern: Virtual Threads — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fan-out 10k blocking HTTP checks
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Virtual Threads: Fan-out 10k blocking HTTP checks

Real-World Scenario

- Running example: Fan-out 10k blocking HTTP checks
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.virtualthreads.problem;

import java.util.*;

public final class NaiveChecks {
  public static int run(int n){ int ok=0; for(int i=0;i<n;i++) ok++; return ok; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.virtualthreads.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class UrlChecker {
  int checkAll(int n) throws Exception {
    try (var scope = Executors.newVirtualThreadPerTaskExecutor()) {
      List<Future<Boolean>> fs=new ArrayList<>();
      for (int i=0;i<n;i++) fs.add(scope.submit(() -> true));
      int ok=0; for (var f: fs) if (f.get()) ok++;
      return ok;
    }
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.virtualthreads.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Virtual Threads: fan-out HTTP checks ===");
    System.out.println("--- naive ---"); System.out.println(NaiveChecks.run(200));
System.out.println("--- virtual threads ---");
System.out.println("checks="+new UrlChecker().checkAll(200));
  }
}
```

Walkthrough Output

```
=== Virtual Threads: fan-out HTTP checks ===
--- naive ---
200
--- virtual threads ---
checks=200
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Virtual Threads.

Related Patterns

- Contrast Virtual Threads with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Virtual Threads → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.